

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Quiz 04 CSE470:Software Engineering Course

Semester: Spring 2023

Duration: 10 Minutes

Full Marks: 15

No. of Questions: 2

No. of Page(s): 2

Name: (Please write in CAPITAL LETTERS)	ID: Sec:
--	-----------------

Answer the following questions.

Which design pattern is used when an object must be created only once? A. Adapter Pattern B. Singleton Pattern C. Bridge Pattern D. Observer Pattern	Which design pattern is used to provide a simplified interface to a complex system of classes? A. Facade Pattern B. Observer Pattern C. Decorator Pattern D. Adapter Pattern
Which design pattern allows an object to be notified automatically of changes in another object's state? A. Decorator Design Pattern B. Observer Design Pattern C. Command Design Pattern D. Adapter Design Pattern	Which design pattern is used to convert the interface of a class into another interface that clients expect? Adapter Design Pattern
In Observer Design Pattern, which component is responsible for maintaining a list of all the observers and notifying them of any changes in the state? A. Subject B. Observer C. ConcreteSubject D. ConcreteObserver	In Adapter Design Pattern, what is the name of the class that adapts the interface of the adaptee to the target interface? A. Adapter B. Adaptee C. Target D. Client
Which design pattern provides a simplified interface to a complex subsystem? A. Facade Design Pattern B. Observer Design Pattern C. Singleton Design Pattern D. Adapter Design Pattern	In Facade Design Pattern, what is the name of the class that provides the simplified interface to the complex subsystem? A. Facade B. Subsystem C. Client D. Adapter

a.

```
class Facade{
    subsystemA: SubsystemA
    subsystemB: SubsystemB
    subsystemC: SubsystemC

    constructor() {
        X = new SubsystemA()
        Y = new SubsystemB()
        Z = new SubsystemC()
    }

    // method to provide a simplified interface to the complex subsystem
    operation() {
        X.operationA()
        Y.operationB()
        Z.operationC()
    }
}
```

b.

```
class Singleton {
    private static Singleton instance;

    private Singleton() {

        //Any other answer for this will be accepted, except for "Singleton"
        public static Singleton getInstance() {
            if (instance == null) {
                instance = new Singleton();
            }
            return instance;
        }
    }
}
```